



Programming Guide

(((

< I >

< } >



< } >



جدول المحتويات

.....	1. مقدمة في عالم البرمجة
.....	2. المتغيرات وأنواع البيانات
.....	3. التعبيرات الشرطية والحلقات
.....	4. الدوال والوحدات البرمجية
.....	5. هياكل البيانات الأساسية
.....	6. البرمجة كائنية التوجه
.....	7. التعامل مع الملفات والبيانات

الفصل 1

مقدمة في عالم البرمجة

البرمجة هي فن وعلم إعطاء تعليمات محددة ودقيقة للحاسوب لتنفيذ مهام معينة. يمكن تشبيه البرمجة بلغة التواصل بين الإنسان والآلة، حيث نستخدم لغات برمجية خاصة لترجمة أفكارنا وتحليلاتنا إلى أوامر يفهمها الحاسوب وينفذها بدقة عالية. تتكون كل لغة برمجية من مجموعة من القواعد النحوية والبنية اللغوية التي تحدد كيفية كتابة الأوامر والتعليمات بشكل صحيح، تماما كما تحدد قواعد اللغة العربية كيفية تكوين الجمل الصحيحة.

تعود جذور البرمجة إلى منتصف القرن التاسع عشر عندما ابتكرت آدا لوفليس أول خوارزمية مكتوبة مصممة لتنفيذ على آلة تحليلية. ومع تطور الحاسوب الإلكتروني في الأربعينيات والخمسينيات من القرن العشرين، ظهرت أولى لغات البرمجة مثل Assembly و Fortran، التي أصبحت أساسا للعديد من اللغات الحديثة C شهدت البرمجة ثورة حقيقية في السبعينيات مع ظهور لغة COBOL و Fortran. ثم جاءت الثمانينيات والتسعينيات مع ++C و Java و Python التي سهلت بشكل كبير عملية البرمجة وجعلتها في متناول Millions من المطورين حول العالم.

تتعدد لغات البرمجة وتختلف حسب استخدامها وأهدافها. تستخدم Python في الذكاء الاصطناعي وتحليل البيانات بسبب بساطتها وقوتها، بينما تعد JavaScript اللغة الأساسية لتطوير تطبيقات الويب التفاعلية. تتميز Java بقابليتها للعمل على مختلف الأنظمة، مما يجعلها خيارا شائعا في تطبيقات المؤسسات الكبيرة. أما ++C فتستخدم في تطوير الألعاب وأنظمة التشغيل والتطبيقات التي تتطلب أداء عاليا. تستخدم Swift لتطوير تطبيقات أجهزة Apple، و Kotlin لتطوير تطبيقات Android.

تحتل البرمجة مكانة محورية في عالمنا المعاصر وتعد من أهم المهارات التي يمكن لأي شخص تعلمها. فكل ما نستخدمه يوميا من تطبيقات الهاتف ومواقع التواصل الاجتماعي ومحركات البحث والألعاب الإلكترونية هو نتاج عمل المبرمجين. تشير الدراسات الحديثة إلى أن الطلب على المبرمجين والمطورين يزداد بشكل مستمر، حيث يتوقع أن يضاف ملايين الوظائف الجديدة في مجال التكنولوجيا خلال العقد القادم. هذا يجعل البرمجة استثمارا ذكيا في المستقبل المهني لأي شخص.

يتيح تعلم البرمجة فرصا وظيفية متنوعة ومرتبعة عالية. يمكن للمبرمج العمل كمطور تطبيقات ويب أو مطور تطبيقات الجوال أو مهندس برمجيات أو محلل بيانات أو متخصص في أمن المعلومات. كما يمكن العمل كمبرمج مستقل من خلال منصات العمل الحر مثل Upwork و Freelancer، مما يوفر مرونة كبيرة في اختيار ساعات العمل والمشاريع. تتراوح الرواتب في هذا المجال بين مستويات جيدة وممتازة حسب الخبرة والتخصص والبلد، وتزداد بشكل ملحوظ مع اكتساب خبرة عملية.

للبدء في تعلم البرمجة، ينصح المبتدئون بالبداية بلغة Python لسهولة تعلمها وقرب بنيتها من اللغة الإنجليزية العادية. يجب أن يركز المتعلم على فهم المفاهيم الأساسية مثل المتغيرات والحلقات والدوال قبل الانتقال إلى مفاهيم أكثر تعقيدا. التواظب على التدريب العملي وحل المشكلات البرمجية يعد أساسا في بناء مهارات البرمجة. توجد منصات تعليمية مجانية مثل Codecademy و Khan Academy و freeCodeCamp التي توفر دورات تفاعلية ممتازة للمبتدئين. الأهم من كل ذلك هو بناء مشاريع عملية تطبق ما freeCodeCamp و Khan Academy تعلمه، لأن البرمجة مهارة تطبيقية بالدرجة الأولى.

المتغيرات وأنواع البيانات

المتغيرات هي من أهم المفاهيم الأساسية في البرمجة، فهي بمثابة حاويات أو صناديق نضع فيها القيم والبيانات التي نحتاجها في برنامجنا. يمكن تخيل المتغير كعلامة اسمية تعلق على قيمة معينة في ذاكرة الحاسوب، مما يتيح لنا الوصول إلى هذه القيمة واستخدامها وتعديلها في أي وقت أثناء تنفيذ البرنامج. عند إنشاء متغير في معظم لغات البرمجة، نحتاج إلى تحديد اسم له وقيمة ابتدائية، وقد نحتاج أيضا إلى تحديد نوع البيانات التي سيجعلها.

أنواع البيانات الأساسية تشمل الأعداد الصحيحة التي تمثل الأرقام الكاملة بدون كسور مثل 5 و 100 و -3، والأعداد العشرية التي تمثل الأرقام ذات الكسور مثل 3.14 و 99.9. نوع بيانات النصوص أو السلاسل الحرفية يستخدم لتخزين النصوص ويوضع عادة بين علامتي تنصيص، مثل اسم الشخص أو عنوانه. النوع المنطقي يحمل إحدى قيمتين فقط: صحيح أو خاطئ، ويستخدم بكثرة في التعبيرات الشرطية والتحقق من الشروط.

تختلف لغات البرمجة في طريقة تعاملها مع المتغيرات. بعض اللغات مثل Python و JavaScript هي لغات ذات ديناميكية في كتابة الأنواع، أي أن المتغير يمكنه تغيير نوعه تلقائيا أثناء تنفيذ البرنامج. بينما لغات مثل Java و ++C تتطلب تحديد نوع المتغير صراحة عند إنشائه ولا يمكن تغييره لاحقا. فمثلا في Python يمكننا كتابة `name = 'أحمد'` ثم `name = 25` في نفس البرنامج، بينما في Java يجب أن نحدد النوع مسبقا مثل `String name = 'أحمد'`.

العمليات الحسابية تشكل جزءا أساسيا من التعامل مع المتغيرات. تتضمن هذه العمليات الجمع والطرح والضرب والقسمة وباقي القسمة وقوة العدد. يمكن استخدام هذه العمليات لإنشاء تعابير رياضية معقدة. بالإضافة إلى العمليات الحسابية، توجد عمليات المقارنة التي تقارن بين قيمتين وترجع نتيجة منطقية، مثل أكبر من وأصغر من وتساوي ولا يساوي. كذلك توجد العمليات المنطقية التي تجمع بين الشروط مثل و أو و لا.

تحويل أنواع البيانات هو عملية تغيير نوع البيانات من نوع إلى آخر، وهو مهم جدا في البرمجة العملية. يمكن تحويل نص رقمي إلى عدد صحيح أو عشري للقيام بعمليات حسابية عليه، كما يمكن تحويل عدد إلى نص لعرضه للمستخدم. بعض التحويلات تتم تلقائيا بواسطة لغة البرمجة، بينما تتطلب تحويلات أخرى استدعاء دوال خاصة. على سبيل المثال، عندما يقوم المستخدم بإدخال رقم عبر لوحة المفاتيح، يكون الرقم في الأصل نصا ويجب تحويله إلى رقم قبل إجراء أي عمليات حسابية عليه.

تسمية المتغيرات لها قواعد يجب اتباعها في كل لغة برمجة. بشكل عام، يجب أن يبدأ اسم المتغير بحرف أو علامة `underscore`، ولا يمكن أن يبدأ برقم. يفضل استخدام أسماء وصفية توضح الغرض من المتغير بدلا من أسماء غامضة مثل `x` أو `y`. فمثلا `salary` أفضل من `s` و `userName` أفضل من `n`. تتبع معظم لغات البرمجة أسلوب `camelCase` حيث تبدأ الكلمة الأولى بحرف صغير وتبدأ كل كلمة تالية بحرف كبير، مثل `totalPrice` أو `firstName`. فهم هذه المفاهيم الأساسية يشكل الأساس المتين الذي يبنى عليه بقية مهارات البرمجة.

التعابير الشرطية والحلقات

التعابير الشرطية هي أدوات أساسية في البرمجة تتيح للبرنامج اتخاذ قرارات بناء على شروط معينة. يمكن تشبيه التعابير الشرطية بعلامات المرور في الطريق، حيث يفحص البرنامج شرطا معيناً ويقرر بناء على نتيجة الفحص أي مسار يجب اتباعه. أبسط شكل للتعبير الشرطية هو عبارة if التي تتحقق من شرط واحد، وإذا كان الشرط صحيحاً تنفذ التعليمات الموجودة داخلها. أما عبارة if-else فتوفر بديلاً آخر ينفذ عندما يكون الشرط خاطئاً.

التعابير الشرطية المتداخلة أو المتعددة تستخدم عند الحاجة للتحقق من عدة شروط مختلفة. يمكن استخدام elif أو else if لإضافة شروط إضافية يفحصها البرنامج بالترتيب حتى يجد شرطاً صحيحاً. يمكن أيضاً دمج عدة شروط في شرط واحد باستخدام العوامل المنطقية and و or و not. فمثلاً يمكننا كتابة شرط يتحقق من أن عمر المستخدم أكبر من 18 وأنه يمتلك رخصة قيادة في نفس الوقت، مما يجعل البرنامج أكثر مرونة وقدرة على التعامل مع حالات مختلفة.

المشغل الشرطي الثلاثي هو اختصار للتعبير الشرطية البسيطة ويكتب في سطر واحد. يتكون من ثلاثة أجزاء: الشرط والقيمة إذا كان الشرط صحيحاً والقيمة إذا كان خاطئاً. يستخدم هذا المشغل عندما نحتاج لتعيين قيمة بناء على شرط بسيط، مما يجعل الكود أكثر إيجازاً وسهولة في القراءة. فمثلاً بدلاً من كتابة عدة أسطر لتحديد ما إذا كان الطالب ناجحاً أو راسباً، يمكننا استخدام المشغل الثلاثي في سطر واحد.

الحلقات هي بنى برمجية تتيح تنفيذ مجموعة من التعليمات بشكل متكرر حتى يتحقق شرط معين أو يتم المرور على جميع عناصر مجموعة بيانات. تعد الحلقات من أهم المفاهيم البرمجية لأنها توفر الكثير من الوقت والجهد في كتابة الكود، حيث بدلاً من كتابة نفس التعليمات مئة مرة يمكننا استخدام حلقة تنفذها مئة مرة تلقائياً. حلقة for تستخدم عندما نعرف مسبقاً عدد مرات التكرار، بينما حلقة while تستخدم عندما نريد التكرار طالما أن شرطاً معيناً محققاً.

حلقة for تتميز بأنها المرور على عناصر مجموعة بيانات واحدة تلو الأخرى. يمكن استخدامها للمرور على عناصر قائمة أو نطاق أرقام أو حتى حروف نص. في Python نكتب for item in list لمعالجة كل عنصر على حدة. حلقة while تستمر في التكرار طالما أن الشرط المحدد يقيم إلى صحيح، ويجب الانتباه لضمان وجود شرط إيقاف مناسب لتجنب الحلقات اللانهائية التي قد تتسبب في تعطل البرنامج. يجب أن تتضمن حلقة while تعليمة تغير قيمة الشرط عاجلاً أم آجلاً.

تعليمات break تستخدم للخروج من الحلقة فوراً عند تحقيق شرط معين، بغض النظر عن عدد التكرارات المتبقية. أما تعليمة continue فتستخدم لتخطي التكرار الحالي والانتقال إلى التكرار التالي. تفيد هاتان التعليمتان في حالات كثيرة، مثل البحث عن continue عنصر في قائمة ووقف البحث فور إيجاد، أو تخطي المعالجة لعناصر لا تستوفي شروطاً معينة. فهم التعابير الشرطية والحلقات واستخدامها بكفاءة هو مفتاح كتابة برامج ذكية وقوية.

الحلقات المتداخلة هي حلقات داخل حلقات أخرى، وتستخدم للتعامل مع بيانات ثنائية الأبعاد مثل المصفوفات والجداول. تتكرر الحلقة الداخلية بالكامل لكل تكرار من الحلقة الخارجية. فمثلاً لطباعة جدول ضرب من 1 إلى 10، نحتاج لحلقة خارجية للأرقام من 1 إلى 10 وحلقة داخلية أيضاً للأرقام من 1 إلى 10. يجب مراعاة أن الحلقات المتداخلة تزيد من وقت التنفيذ بشكل كبير مع زيادة حجم البيانات، لذا يجب استخدامها بحكمة وتحسين أدائها عند الحاجة.

الدوال والوحدات البرمجية

الدوال هي كتل برمجية مغلقة تؤدي مهمة محددة ويمكن استدعاؤها في أي وقت من البرنامج. تعد الدوال من أهم المفاهيم في البرمجة لأنها تساعد على تنظيم الكود وتقسيمه إلى أجزاء صغيرة يسهل فهمها وصيانتها وإعادة استخدامها. يمكن تشبيه الدالة بآلة مصغرة تأخذ مدخلات معينة وتعالجها وتخرج نتيجة محددة. كل دالة لها اسم فريد يستخدم لاستدعائها، وقد تحتوي على معاملات وقيمة إرجاعية.

تعريف الدالة يتضمن عادة الكلمة المفتاحية `def` في Python أو `function` في JavaScript، يليها اسم الدالة وقوسان يحتويان على المعاملات الاختيارية. المعاملات هي قيم نمررها للدالة عند استدعائها لاستخدامها في عملياتها الداخلية. يمكن أن تكون المعاملات إلزامية يجب توفيرها عند الاستدعاء، أو اختيارية لها قيم افتراضية تستخدم عند عدم تمرير قيمة. يمكن للدالة أن ترجع قيمة باستخدام الكلمة المفتاحية `return`، وإذا لم ترجع قيمة فإنها ترجع لا شيء افتراضياً.

تصنف الدوال حسب نطاقها إلى دوال عامة يمكن استدعاؤها من أي مكان في البرنامج، ودوال محلية لا يمكن الوصول إليها إلا من داخل الدالة أو الكتلة التي عرفت فيها. نطاق المتغيرات هو مفهوم مهم مرتبط بالدوال، حيث أن المتغيرات المعرفة داخل دالة تكون محلية بها ولا يمكن الوصول إليها من خارجها، بينما المتغيرات المعرفة خارج أي دالة تكون عامة ويمكن الوصول إليها من جميع أنحاء البرنامج. فهم نطاق المتغيرات يساعد في تجنب الأخطاء البرمجية وكتابة كود أكثر أماناً.

البرمجة الوظيفية هي نمط برمجي يعتمد على الدوال كعنصر أساسي في البناء. في هذا النمط، تعامل الدوال كقيم يمكن تمريرها كمعاملات لدوال أخرى أو إرجاعها كنتائج. الدوال المجهولة أو `Lambda` هي دوال صغيرة بدون اسم تستخدم عادة لعمليات بسيطة. الدوال العودية هي دوال تستدعي نفسها لحل مشكلة ما عن طريق تقسيمها إلى مشاكل أصغر حتى الوصول إلى حالة أساسية بسيطة. تستخدم الدوال العودية بكثرة في خوارزميات البحث والترتيب.

الوحدات البرمجية أو المكتبات هي ملفات تحتوي على مجموعة من الدوال والكودات الجاهزة التي يمكن استيرادها واستخدامها في برامجنا. توفر لغات البرمجة مكتبات قياسية غنية تشمل دوال للتعامل مع التواريخ والملفات والأرقام والنصوص والشبكات وغيرها الكثير. بالإضافة إلى المكتبات القياسية، توجد مكتبات خارجية أنشأها مطورون من جميع أنحاء العالم ويمكن تثبيتها واستخدامها مجاناً. في Python نستخدم `pip` لتثبيت المكتبات الخارجية، بينما في JavaScript نستخدم `npm`.

من المكتبات الشهيرة في Python مكتبة `requests` للتعامل مع طلبات الشبكة، ومكتبة `pandas` لتحليل البيانات، ومكتبة `numpy` للعمليات الحسابية المتقدمة، ومكتبة `matplotlib` لإنشاء الرسوم البيانية. في JavaScript تعد مكتبة `React` أساسية لبناء واجهات المستخدم، ومكتبة `Express` لتطوير الخوادم، ومكتبة `Lodash` للعمليات المساعدة. استخدام المكتبات الجاهزة يوفر الكثير من الوقت والجهد ويضمن استخدام كود مختبر وموثوق، لكن يجب فهم كيفية عملها لاستخدامها بشكل صحيح.

لتصميم دالة جيدة، يجب أن تكون مسؤولة عن مهمة واحدة فقط واضحة ومحددة. يجب أن يكون اسمها وصفيًا يوضح ما تفعله، وأن تكون معاملاتها ذات أسماء واضحة. يجب أيضاً كتابة تعليقات توضيحية تشرح الغرض من الدالة ومعاملاتها وقيمتها إرجاعها. يجب أن تكون الدالة قصيرة قدر الإمكان، حيث القاعدة العامة تقول إذا كانت الدالة لا تسع في شاشة واحدة فقد تحتاج إلى تقسيمها. استخدام الدوال بكفاءة هو علامة على مبرمج محترف ينتج كوداً نظيفاً ومحافظةً عليه.

هياكل البيانات الأساسية

هياكل البيانات هي طرق منظمة لتخزين البيانات وإدارتها بكفاءة داخل البرنامج. فهم هياكل البيانات وتقنياتها يعد من المهارات الأساسية لأي مبرمج يرغب في كتابة برامج فعالة وسريعة. كل هيكل بيانات له مميزات وعيوبه ويكون مناسباً لأنواع معينة من العمليات. الاختيار الصحيح لهيكل البيانات يمكن أن يحدث فرقاً كبيراً في أداء البرنامج، خاصة عند التعامل مع كميات كبيرة من البيانات.

المصفوفات أو القوائم هي أبسط وأكثر هياكل البيانات استخداماً. تخزن المصفوفة مجموعة من العناصر بجانب بعضها في الذاكرة، ويمكن الوصول إلى أي عنصر من خلال فهرسه أو رقمه الترتيبي. تبدأ الفهارس من الصفر في معظم لغات البرمجة. تتيح المصفوفات إضافة عناصر جديدة في نهايتها، وحذف عناصر، وتعديل قيم عناصر موجودة، والبحث عن عناصر معينة. من عيوب المصفوفات أن حجمها قد يكون ثابتاً في بعض اللغات، وأن إدراج عنصر في المنتصف يتطلب إزاحة جميع العناصر التالية.

القوائم المرتبطة هي هياكل بيانات بديل للمصفوفات يتميز بالمرونة في إضافة وحذف العناصر. في القائمة المرتبطة، لا يخزن كل عنصر بجانب الآخر في الذاكرة، بل يحتوي كل عنصر على قيمته و رابط يشير إلى العنصر التالي. هذا يسهل إدراج وحذف العناصر في أي مكان دون الحاجة لإزاحة العناصر الأخرى، لكن الوصول إلى عنصر في منتصف القائمة يتطلب المرور على جميع العناصر السابقة له من البداية، مما يجعل البحث أبطأ مقارنة بالمصفوفات.

المكدس هو هيكل بيانات يتبع مبدأ آخر ما يدخل أول ما يخرج. يشبه المكدس مجموعة من الأطباق المترصة، حيث نضع الطبق الجديد فوق الكومة ونأخذ الطبق العلوي أولاً. العمليات الأساسية على المكدس هي الدفع لإضافة عنصر في الأعلى، والسحب لإزالة العنصر العلوي، والنظر للاطلاع على العنصر العلوي دون إزالته. يستخدم المكدس في تطبيقات مثل تتبع استدعاءات الدوال، والتراجع عن العمليات في محررات النصوص، وتقييم التعبيرات الحسابية.

الطابور هو هيكل بيانات يتبع مبدأ أول ما يدخل أول ما يخرج. يشبه الطابور صف الانتظار في المحل التجاري، حيث يدخل الشخص أولاً ويخرج أولاً. العمليات الأساسية هي الإدراج في الخلف والإزالة من الأمام. يستخدم الطابور في إدارة مهام الطباعة، ومعالجة الطلبات في أنظمة الخوادم، وانتظار المعالجة في تطبيقات الدردشة. الطابور ذو الأولوية هو نسخة محسنة يخرج العنصر ذا الأولوية الأعلى بغض النظر عن ترتيب دخوله.

القواميس أو الخرائط هي هياكل بيانات تخزن البيانات في شكل أزواج مفتاح-قيمة. كل عنصر يتكون من مفتاح فريد يستخدم للوصول إلى القيمة المقابلة له. تشبه القواميس القاموس العادي حيث نبحث عن كلمة للحصول على معناها. تتميز القواميس بسرعة الوصول إلى القيم عبر المفاتيح، حيث تتم عمليات البحث والإدراج والحذف بسرعة ثابتة تقريباً. تستخدم بكثرة في تطبيقات مثل تخزين معلومات المستخدمين وذاكرة التخزين المؤقت وتكوين التطبيقات.

المجموعات هي هياكل بيانات تخزن عناصر فريدة بدون تكرار. تفيد في إزالة التكرارات من قائمة، والتحقق من وجود عنصر، وعمليات التقاطع والاتحاد بين المجموعات. الأشجار هي هياكل بيانات هرمية تتكون من عقد مرتبطة ببعضها، تبدأ من عقدة جذر وتتفرع إلى عقد فرعية. تستخدم في تمثيل الهياكل التنظيمية ونظم الملفات وحركات البحث. الرسوم البيانية تتكون من عقد وروابط تربط بينها، وتستخدم في تمثيل شبكات التواصل والخرائط وأنظمة التنقل. اختيار هيكل البيانات المناسب يتطلب فهماً لمتطلبات البرنامج من حيث سرعة الوصول وحجم البيانات ونوع العمليات المطلوبة.

البرمجة كائنية التوجه

البرمجة كائنية التوجه هي نمط برمجي يعتمد على مفهوم الكائنات التي تجمع بين البيانات والسلوكيات في وحدة واحدة متماسكة. هذا النمط يحاكي الطريقة التي ندرك بها العالم الحقيقي، حيث كل شيء حولنا هو كائن له صفات وسلوكيات. فالسيارة على سبيل المثال لها صفات كاللون والسرعة والموديل، وسلوكيات كالتسارع والفرملة والانعطاف. تنظم البرمجة كائنية التوجه الكود بشكل يسهل فهمه وتطويره وصيانتته وإعادة استخدام مكوناته.

الفئة أو التصنيف هي القالب أو النموذج الذي نصمم من خلاله الكائنات. تحدد الفئة الصفات والسلوكيات التي ستكون موجودة في كل كائن ينشأ منها. الكائن هو نسخة فعلية من الفئة موجودة في الذاكرة. فمثلا يمكننا تعريف فئة باسم طالب تحتوي على صفات مثل الاسم والعمر والدرجات، وسلوكيات مثل الدراسة والتقديم. ثم ننشئ عدة كائنات من هذه الفئة يمثل كل منها طالبا حقيقيا بقيمه الخاصة.

الوراثة هي أحد أهم مفاهيم البرمجة كائنية التوجه، وتسمح بإنشاء فئة جديدة تستفيد من صفات وسلوكيات فئة موجودة مسبقا. الفئة الجديدة تسمى الفئة الفرعية والفئة الأصلية تسمى الفئة الأساسية. هذا يقلل من تكرار الكود ويسهل الصيانة. فمثلا يمكننا تعريف فئة حيوان عامة تحتوي صفات مشتركة بين جميع الحيوانات، ثم ننشئ فئات فرعية مثل قطة وكلب ترث من الحيوان وتضيف صفات وسلوكيات خاصة بها. الفئة الفرعية يمكنها أيضا تجاوز السلوكيات الموروثة وتوفير تنفيذ مختلف لها.

التغليف هو مبدأ يقضي بإخفاء التفاصيل الداخلية للكائن وجعل الوصول إلى بياناته من خلال واجهات محددة. هذا يحمي البيانات من التعديل المباشر غير المرغوب ويجعل الكود أكثر أمانا ومرونة. يتم تطبيق التغليف باستخدام معدلات الوصول مثل خاص ومحمي وعام. البيانات الخاصة لا يمكن الوصول إليها إلا من داخل الكائن نفسه، بينما البيانات المحمية يمكن الوصول إليها من الكائن والفئات الفرعية، والبيانات العامة يمكن الوصول إليها من أي مكان.

تعدد الأشكال هو قدرة الكائنات من فئات مختلفة على الاستجابة لنفس الاستدعاء بطرق مختلفة. هذا يتيح التعامل مع أنواع مختلفة من الكائنات من خلال واجهة موحدة. فمثلا إذا كان لدينا دالة اسمها makeSound، يمكنها أن تنفذ بشكل مختلف على كائن كلب فينبج، وعلى كائن قطة فتواء، وعلى كائن طائر فيغرد. هذا المفهوم يسهل إضافة أنواع جديدة دون تعديل الكود الموجود، مما يجعل البرنامج أكثر قابلية للتوسع.

الواجهات والفئات المجردة هي أدوات إضافية في البرمجة كائنية التوجه. الواجهة تحدد مجموعة من السلوكيات التي يجب أن تنفذها أي فئة تتبناها، دون تحديد كيفية التنفيذ. الفئة المجردة هي فئة لا يمكن إنشاء كائنات منها مباشرة، وتحتوي على سلوكيات منفذة وأخرى مجردة يجب على الفئات الفرعية تنفيذها. هذه المفاهيم تساعد في بناء هياكل برمجية منظمة ومرنة.

مبدأ المسؤولية الواحدة ينص على أن كل فئة يجب أن تكون مسؤولة عن مهمة واحدة فقط. مبدأ الافتقار إلى المعرفة ينص على أن الكائنات يجب أن تفهم أقل ما يكفي عن بعضها البعض. مبدأ استبدال iskov ينص على أن الفئة الفرعية يجب أن تكون قابلة للاستبدال بالفئة الأساسية دون التأثير على صحة البرنامج. هذه المبادئ معا تشكل أسس التصميم الجيد كائني التوجه وتضمن إنتاج برامج قابلة للصيانة والتطوير على المدى الطويل.

التعامل مع الملفات والبيانات

يعد التعامل مع الملفات والبيانات من المهارات الأساسية التي يحتاجها كل مبرمج، لأن معظم التطبيقات تحتاج إلى قراءة البيانات من مصادر خارجية وحفظ نتائجها بشكل دائم. بدون القدرة على تخزين البيانات، ستفقد جميع المعلومات فور إغلاق البرنامج. يوفر نظام التشغيل آليات متعددة للتعامل مع الملفات، وتدعم لغات البرمجة مكتبات وأدوات خاصة لقراءة وكتابة الملفات بأنواعها المختلفة.

عمليات الملفات الأساسية تشمل فتح الملف للقراءة أو الكتابة أو الإلحاق، وقراءة محتواه، وكتابة بيانات جديدة فيه، وإغلاقه بعد الانتهاء. عند فتح ملف يجب دائماً استخدام عبارة إدارة السياق أو كتلة try-finally لضمان إغلاق الملف حتى في حالة حدوث خطأ أثناء المعالجة. يمكن قراءة الملف بالكامل دفعة واحدة، أو قراءته سطراً بسطراً، وهو الأسلوب المفضل عند التعامل مع ملفات كبيرة لتجنب استهلاك الذاكرة. يمكن أيضاً الكتابة على الملف بالكامل دفعة واحدة أو إلحاق بيانات بنهايته دون مسح المحتوى الحالي.

توجد عدة صيغ شائعة للملفات النصية، أشهرها صيغة CSV التي تخزن البيانات الجدولية حيث كل سطر يمثل سجلاً والحقول مفصولة بفاصلة. هذه الصيغة بسيطة ومدعومة من معظم التطبيقات ويمكن فتحها في أي محرر نصوص أو برنامج جداول بيانات. صيغة JSON أصبحت المعيار الأكثر استخداماً لتبادل البيانات على الويب، حيث تخزن البيانات في شكل أزواج مفتاح-قيمة وقوائم، وتدعم هياكل بيانات متداخلة ومعقدة. تدعم معظم لغات البرمجة تحويل البيانات من وإلى JSON تلقائياً.

قواعد البيانات هي أنظمة متخصصة في تخزين البيانات وإدارتها بكفاءة واسترجاعها بسرعة. تختلف عن الملفات العادية في أنها توفر آليات متقدمة للاستعلام والفهرسة والتحكم بالصلاحيات والتعامل مع البيانات المترامنة. قواعد البيانات العلائقية مثل MySQL و PostgreSQL للاستعلام عن البيانات. قواعد SQL تخزن البيانات في جداول مرتبطة ببعضها عبر مفاتيح أجنبية، وتستخدم لغة PostgreSQL للبيانات غير العلائقية مثل MongoDB تخزن البيانات في شكل وثائق مرنة دون الحاجة لهيكل ثابت مسبقاً.

لغة SQL هي المعيار الأساسي للتعامل مع قواعد البيانات العلائقية. تتضمن أوامر SQL للتعريف مثل CREATE و ALTER و لاستعلام DELETE و UPDATE و INSERT و SELECT لإنشاء وتعديل وحذف الجداول. أوامر التعامل مع البيانات مثل DROP وإدراج وتعديل وحذف البيانات. عبارة SELECT هي الأكثر استخداماً وتتيح تصفية البيانات وترتيبها وتجميعها باستخدام تعابير شرطية وعمليات حسابية. يمكن ربط عدة جداول معاً باستخدام عبارات JOIN للحصول على بيانات مدمجة من مصادر متعددة.

واجهات برمجة التطبيقات أو APIs هي آلية تتيح للتطبيقات التواصل فيما بينها وتبادل البيانات. تعتمد معظم واجهات الويب الحديثة على معمارية REST التي تستخدم بروتوكول HTTP لتبادل البيانات بصيغة JSON. يمكن للتطبيق إرسال طلبات GET للحصول على بيانات، و POST لإرسال بيانات جديدة، و PUT أو PATCH لتحديث بيانات موجودة، و DELETE لحذفها. يرجع الخادم استجابة تتضمن رمز حالة يشير إلى نجاح أو فشل العملية، والبيانات المطلوبة إن وجدت.

الأمان هو جانب حاسم في التعامل مع الملفات والبيانات. يجب دائماً التحقق من صلاحية الملف قبل محاولة فتحه، ومعالجة البيانات المدخلة من المستخدم لتجنب هجمات الحقن. في قواعد البيانات، يجب استخدام معاملات معدة بدلاً من تضمين البيانات مباشرة في استعلامات SQL لتجنب هجمات حقن SQL. يجب تشفير البيانات الحساسة قبل تخزينها، واستخدام اتصالات مشفرة عند نقل البيانات عبر الشبكة. نسخ البيانات احتياطياً بشكل منتظم إجراء ضروري لحمايتها من الفقدان. هذه الممارسات الأمنية ليست اختيارية بل ضرورة في أي تطبيق حقيقي يتعامل مع بيانات المستخدمين.

